Notación Big O

Teoría de Grafos 2026
Miguel Condori

Universidad Mayor de San Simón

29 de agosto de 2026

¿Por qué estudiar complejidad?

- Nos permite estimar cómo crece el **número de operaciones** de un algoritmo cuando aumenta la entrada.
- No medimos segundos exactos, medimos crecimiento asintótico.
- En teoría de grafos es vital para comparar algoritmos sobre muchos vértices o aristas.
- La notación Big O da un límite superior (peor caso) que nos garantiza comportamiento en situaciones adversas.

Idea central de Big O

- Big O es una forma de *clasificar* el crecimiento del costo (operaciones) de un algoritmo.
- Usamos el **peor caso** — el escenario donde se realizan más operaciones.
- Con Big O decimos: *cómo* crece el costo cuando el tamaño de la entrada aumenta.

¿Por qué estudiar el peor caso?

- Porque proporciona una **garantía** — el algoritmo no hará más operaciones que ese límite.
- Es útil en grafos cuando queremos seguridad aun en entradas adversas.
- Si controlamos el peor caso, todos los demás casos están cubiertos.

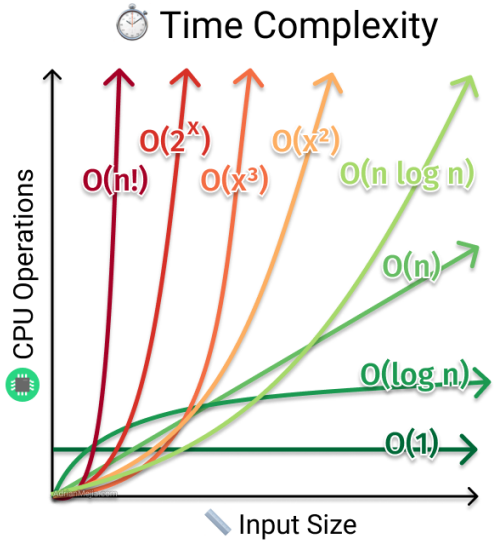
Definición intuitiva de O(n)

- Decimos que un algoritmo es O(n) cuando el número de operaciones crece **de forma proporcional** al tamaño de la entrada.
- Este crecimiento se llama **lineal**.

Pasos para estimar O(n)

1. Identificar claramente qué representa n (vértices, aristas, tamaño de una lista...).
2. Contar cuántas veces se ejecuta la operación principal en el peor caso.
3. Sumar fases secuenciales (se suman sus costos).
4. Ignorar constantes multiplicativas y sumandos de menor orden.

Comparación de Complejidades



Ejemplo 1: Recorrer un arreglo

```
public void imprimir(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        System.out.println(arr[i]);
    }
}
```

- Si el arreglo tiene n elementos, el ciclo se ejecuta n veces.
- Cada iteración hace una operación constante.
- Total de operaciones proporcional a n.
- Complejidad en el peor caso: O(n).

Ejemplo 1: Complejidad O(1)

```
public int obtenerPrimero(int[] arr) {
    return arr[0];
}
```

- No depende del tamaño del arreglo.
- Siempre ejecuta una sola operación.
- Aunque el arreglo tenga 10 o 1 millón de elementos.
- Complejidad: O(1).

Ejemplo 2: O(n) con constante

```
public void imprimir(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        System.out.println(arr[i]);
    }
}
```

- Se ejecuta n veces.
- Supongamos 3 operaciones internas.
- Total aproximado: 3n.
- En Big O ignoramos constantes $\rightarrow$ O(n).

Ejemplo 3: 5 * $O(n^2)$

```
public void ejemplo(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < 5; k++) {
                System.out.println(i + j + k);
            }
        }
    }
}
```

- Primer ciclo: n
- Segundo ciclo: n
- Tercer ciclo: 5 (constante)
- Total aproximado: $5n^2$
- Big O elimina constante 5 $\rightarrow$ $O(n^2)$

Ejemplo 4: $O(n^2)$

```
public void pares(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.println(i + "," + j);
        }
    }
}
```

- $n \times n$ iteraciones.
- Total: n^2.
- Complejidad: $O(n^2)$.

Ejemplo 5: $O(n^3)$

```
public void triple(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                System.out.println(i + j + k);
            }
        }
    }
}
```

- $n \times n \times n$ iteraciones.
- Total: n^3.
- Complejidad: $O(n^3)$.

Ejemplo 6: Mezcla $n^2 + n$

```
public void mezcla(int n) {

    for (int i = 0; i < n; i++) {
        System.out.println(i);
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.println(i + j);
        }
    }
}
```

- Primer bloque: n
- Segundo bloque: n^2
- Total: $n^2 + n$

Ejemplo 7: $n^2 + n + 1$

```
public void ejemploCompleto(int n) {

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.println(i + j);
        }
    }

    for (int i = 0; i < n; i++) {
        System.out.println(i);
    }

    System.out.println("Fin");
}
```

- Doble ciclo: n^2

Ejercicio 1

```
public void ejercicio1(int n) {
    for (int i = 0; i < n; i++) {
        System.out.println(i);
    }
    for (int j = 0; j < n; j++) {
        System.out.println(j);
    }
}
```

Pregunta: ¿Cuál es la complejidad en el peor caso?

Respuesta Ejercicio 1

- Primer ciclo: n
- Segundo ciclo: n
- Total: 2n
- Ignorando constantes $\rightarrow$ O(n)

Ejercicio 2

```
public void ejercicio2(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.println(i + j);
        }
    }
}
```

Pregunta: ¿Cuál es la complejidad?

Respuesta Ejercicio 2

- $n \times n$ iteraciones.
- Total: n^2.
- Complejidad: $O(n^2)$.

Ejercicio 3

```
public void ejercicio3(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < 10; j++) {
            System.out.println(i + j);
        }
    }
}
```

Pregunta: ¿Es $O(n^2)$ o $O(n)$?

Respuesta Ejercicio 3

- Primer ciclo: n
- Segundo ciclo: 10 (constante)
- Total: 10n
- Ignoramos constante $\rightarrow$ O(n)

Ejercicio 4

```
public void ejercicio4(int n) {

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.println(i + j);
        }
    }

    for (int k = 0; k < n; k++) {
        System.out.println(k);
    }
}
```

Pregunta: ¿Cuál es la complejidad final?

Respuesta Ejercicio 4

- Primer bloque: n^2
- Segundo bloque: n
- Total: $n^2 + n$
- Término dominante: n^2
- Complejidad final: $O(n^2)$

Ejercicio 5

```
public void ejercicio5(int n) {

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < i; j++) {
            System.out.println(i + j);
        }
    }
}
```

Pregunta: ¿Es $O(n^2)$ o algo menor?

Respuesta Ejercicio 5

- El ciclo interno depende de i.
- Total de iteraciones:

$$0 + 1 + 2 + 3 + \cdots + (n-1)$$

- Esto es:

$$\frac{n(n-1)}{2}$$

- Crecimiento cuadrático.
- Complejidad: $O(n^2)$

Ejercicio 6

```
public void ejercicio6(int n) {

    for (int i = 1; i < n; i *= 2) {
        for (int j = 0; j < n; j++) {
            System.out.println(i + j);
        }
    }
}
```

Pregunta: Analice cuidadosamente la complejidad.

Respuesta Ejercicio 6

- El ciclo externo se duplica cada vez:

$$1, 2, 4, 8, \ldots$$

- Se ejecuta aproximadamente log n veces.
- El ciclo interno ejecuta n veces.
- Total: n log n
- Complejidad: O(n log n)

Ejercicio 7

```
public void ejercicio7(int n) {

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < n; j++) {

            for (int k = 0; k < 3; k++) {
                System.out.println(i + j + k);
            }
        }
    }
}
```

Pregunta: ¿Es $O(n^3)$?

Respuesta Ejercicio 7

- Primer ciclo: n
- Segundo ciclo: n
- Tercer ciclo: 3 (constante)
- Total: $3n^2$
- Ignoramos constante $\rightarrow O(n^2)$
- No es $O(n^3)$.

Ejercicio 8

```
public void ejercicio8(int n) {

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < n; j++) {

            for (int k = 0; k < n; k++) {

                for (int t = 0; t < 2; t++) {
                    System.out.println(i+j+k+t);
                }

            }
        }
    }
```

Respuesta Ejercicio 8

- Cuatro ciclos.
- Tres dependen de n.
- Uno es constante (2).
- Total aproximado: $2n^3$
- Ignoramos constante $\rightarrow O(n^3)$